

BOOK A DOCTOR SYSTEM

Create Schemas and Models

The Book a Doctor backend uses **Mongoose Schemas** to define the structure of documents stored in MongoDB. Each schema is converted into a model that performs CRUD operations on the corresponding collection.

1. User Model (userModel.js)

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  fullname: String,
  email: String,
  password: String,
  phone: String,
  role: String
},{timestamps:true});

module.exports = mongoose.model("User", userSchema);
```

Purpose: Stores authentication and profile information for Admins, Doctors and Patients.

2. Doctor Model (doctorModel.js)

```
const doctorSchema = new mongoose.Schema({
  userId:{type:mongoose.Schema.Types.ObjectId,ref:"User"},
  specialization:String,
  qualification:String,
  experience:Number,
  consultationFee:Number,
  hospital:String,
  address:String,
  about:String,
  available:Boolean
},{timestamps:true});

module.exports=mongoose.model("Doctor",doctorSchema);
```

Stores doctor profile information linked to a registered user.

3. Appointment Model (appointmentModel.js)

```
const appointmentSchema = new mongoose.Schema({
  patient:{type:mongoose.Schema.Types.ObjectId,ref:"User"},
  doctor:{type:mongoose.Schema.Types.ObjectId,ref:"Doctor"},
  appointmentDate:Date,
  status:String,
  reason:String
},{timestamps:true});

module.exports=mongoose.model("Appointment",appointmentSchema);
```

Maintains appointment booking details between patients and doctors.

Advantages

- Strong schema validation.
- Automatic timestamps.
- Easy relationship mapping using ObjectId references.
- Simplifies CRUD operations through Mongoose models.
- Keeps the backend modular and maintainable.